

[BUG] Cannot decompose >3 two-qubit unitaries

<https://github.com/PennyLaneAI/pennylane/issues/2014>

ROW 71

[BUG] Cannot decompose >3 two-qubit unitaries #2014

[New issue](#)[Closed](#)

glassnotes opened on Dec 11, 2021 · edited by glassnotes

Edits · ...

Expected behavior

The `unitary_to_rot` transform should be able to decompose an arbitrary number of two-qubit `QubitUnitary` operations.

Actual behavior

A `RecursionError` occurs when attempting to decompose more than 3 such operations.

Additional information

Reproduces always.

Source code

```
import pennylane as qml
from pennylane import numpy as np
from scipy.stats import unitary_group

n_wires = 2
dev = qml.device("default.qubit", wires=n_wires)

random_U_4 = [unitary_group.rvs(4) for _ in range(4)]

@qml.qnode(dev)
@qml.transforms.unitary_to_rot
def my_circuit():
    qml.QubitUnitary(random_U_4[0], wires=[0, 1])
    qml.QubitUnitary(random_U_4[1], wires=[0, 1])
    qml.QubitUnitary(random_U_4[2], wires=[0, 1])

    # If you comment out this next one, it will run without errors
    qml.QubitUnitary(random_U_4[3], wires=[0, 1])

    return qml.expval(qml.PauliZ(0))

print(my_circuit())
```

Tracebacks

```
RecursionError                                Traceback (most recent call last)
~/Code/pennylane/pennylane/transforms/qfunc_transforms.py in __call__(self, tape, *args, **kwargs)
    165         with tape_class() as new_tape:
--> 166             self.transform_fn(tape, *args, **kwargs)
    167

~/Code/pennylane/pennylane/transforms/unitary_to_rot.py in unitary_to_rot(tape)
    138         elif qml.math.shape(op.parameters[0]) == (4, 4):
--> 139             two_qubit_decomposition(op.parameters[0], op.wires)
    140         else:

~/Code/pennylane/pennylane/transforms/decompositions/two_qubit_unitary.py in two_qubit_decomposition(U, wires)
    608         for op in decomp:
--> 609             qml.apply(op)
    610

~/Code/pennylane/pennylane/tape/tape.py in __exit__(self, exception_type, exception_value, traceback)
    354         super().__exit__(exception_type, exception_value, traceback)
--> 355         self._process_queue()
    356         finally:

~/Code/pennylane/pennylane/transforms/qfunc_transforms.py in _process_queue(self)
     83         def _process_queue(self):
--> 84             super()._process_queue()
     85
```

Assignees

No one assigned

Labels

[bug](#)

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications

[Customize](#)[Subscribe](#)

You're not receiving notifications from this thread.

Participants



Zoterc

85

```
~/Code/pennylane/pennylane/tape/tape.py in _process_queue(self)
452
--> 453     self._update()
454

~/Code/pennylane/pennylane/tape/tape.py in _update(self)
510     self._update_circuit_info()
--> 511     self._update_par_info()
512     self._update_trainable_params()

~/Code/pennylane/pennylane/tape/tape.py in _update_par_info(self)
488
--> 489     for p in range(len(obj.data)):
490         info = self._par_info.get(param_count, {})

~/Code/pennylane/pennylane/tape/tape.py in data(self)
1227     for backwards compatibilities with operations."""
-> 1228     return self.get_parameters(trainable_only=False)
1229

~/Code/pennylane/pennylane/tape/tape.py in get_parameters(self, trainable_only, **kwargs)
815     op_idx = self._par_info[p_idx]["p_idx"]
--> 816     params.append(op.data[op_idx])
817     return params

... last 2 frames repeated, from the frame below ...

~/Code/pennylane/pennylane/tape/tape.py in data(self)
1227     for backwards compatibilities with operations."""
-> 1228     return self.get_parameters(trainable_only=False)
1229

RecursionError: maximum recursion depth exceeded while calling a Python object

During handling of the above exception, another exception occurred:

RecursionError                                Traceback (most recent call last)
~/Code/pennylane/pennylane/qnode.py in construct(self, args, kwargs)
493     with self.tape:
--> 494         self._qfunc_output = self.func(*args, **kwargs)
495

~/Code/pennylane/pennylane/transforms/qfunc_transforms.py in internal_wrapper(*args, **kwargs)
197     tape = make_tape(fn)(*args, **kwargs)
--> 198     tape = tape_transform(tape, *transform_args, **transform_kwargs)
199

~/Code/pennylane/pennylane/transforms/qfunc_transforms.py in __call__(self, tape, *args, **kwargs)
165     with tape_class() as new_tape:
--> 166         self.transform_fn(tape, *args, **kwargs)
167

~/Code/pennylane/pennylane/tape/tape.py in __exit__(self, exception_type, exception_value, traceback)
354     super().__exit__(exception_type, exception_value, traceback)
--> 355     self._process_queue()
356     finally:

~/Code/pennylane/pennylane/transforms/qfunc_transforms.py in _process_queue(self)
83     def _process_queue(self):
--> 84     super()._process_queue()
85

~/Code/pennylane/pennylane/tape/tape.py in _process_queue(self)
452
--> 453     self._update()
454

~/Code/pennylane/pennylane/tape/tape.py in _update(self)
510     self._update_circuit_info()
--> 511     self._update_par_info()
512     self._update_trainable_params()

~/Code/pennylane/pennylane/tape/tape.py in _update_par_info(self)
488
--> 489     for p in range(len(obj.data)):
490         info = self._par_info.get(param_count, {})

~/Code/pennylane/pennylane/tape/tape.py in data(self)
1227     for backwards compatibilities with operations."""
-> 1228     return self.get_parameters(trainable_only=False)
1229
```

RecursionError: maximum recursion depth exceeded while calling a Python object

During handling of the above exception, another exception occurred:

```
RecursionError                                Traceback (most recent call last)
/tmp/ipykernel_1021057/2165726760.py in <module>
----> 1 my_circuit()

~/Code/pennylane/pennylane/qnode.py in __call__(self, *args, **kwargs)
    554
    555     # construct the tape
--> 556     self.construct(args, kwargs)
    557
    558     # preprocess the tapes by applying any device-specific transforms

~/Code/pennylane/pennylane/qnode.py in construct(self, args, kwargs)
    492
    493     with self.tape:
--> 494         self._qfunc_output = self.func(*args, **kwargs)
    495
    496         params = self.tape.get_parameters(trainable_only=False)

~/Code/pennylane/pennylane/tape/tape.py in __exit__(self, exception_type, exception_value, traceback)
    353     try:
    354         super().__exit__(exception_type, exception_value, traceback)
--> 355         self._process_queue()
    356     finally:
    357         QuantumTape._lock.release()

~/Code/pennylane/pennylane/tape/tape.py in _process_queue(self)
    451         self.is_sampled = True
    452
--> 453         self._update()
    454
    455     def _update_circuit_info(self):

~/Code/pennylane/pennylane/tape/tape.py in _update(self)
    509         self._depth = None
    510         self._update_circuit_info()
--> 511         self._update_par_info()
    512         self._update_trainable_params()
    513         self._update_observables()

~/Code/pennylane/pennylane/tape/tape.py in _update_par_info(self)
    487         for obj in self.operations + self.observables:
    488
--> 489             for p in range(len(obj.data)):
    490                 info = self._par_info.get(param_count, {})
    491                 info.update({"op": obj, "p_idx": p})

~/Code/pennylane/pennylane/tape/tape.py in data(self)
    1226         """Alias to :meth:`~.get_parameters` and :meth:`~.set_parameters`
    1227         for backwards compatibilities with operations."""
-> 1228         return self.get_parameters(trainable_only=False)
    1229
    1230     @data.setter

~/Code/pennylane/pennylane/tape/tape.py in get_parameters(self, trainable_only, **kwargs)
    814         op = self._par_info[p_idx]["op"]
    815         op_idx = self._par_info[p_idx]["p_idx"]
--> 816         params.append(op.data[op_idx])
    817     return params
    818

... last 2 frames repeated, from the frame below ...

~/Code/pennylane/pennylane/tape/tape.py in data(self)
    1226         """Alias to :meth:`~.get_parameters` and :meth:`~.set_parameters`
    1227         for backwards compatibilities with operations."""
-> 1228         return self.get_parameters(trainable_only=False)
    1229
    1230     @data.setter
```

RecursionError: maximum recursion depth exceeded while calling a Python object

System information

Name: PennyLane
Version: 0.21.0.dev0
Summary: PennyLane is a Python quantum machine learning library by Xanadu Inc.
Home-page: <https://github.com/XanaduAI/pennylane>



RecursionError: maximum recursion depth exceeded while calling a Python object

System information

```
Name: PennyLane
Version: 0.21.0.dev0
Summary: PennyLane is a Python quantum machine learning library by Xanadu Inc.
Home-page: https://github.com/XanaduAI/pennylane
Author:
Author-email:
License: Apache License 2.0
Location: /home/olivia/Code/pennylane
Requires: numpy, scipy, networkx, autograd, toml, appdirs, semantic_version, autoray, cachetools, pennylane-lightning
Required-by: PennyLane-Lightning
Platform info: Linux-5.11.0-41-generic-x86_64-with-glibc2.31
Python version: 3.9.7
Numpy version: 1.21.0
Scipy version: 1.7.0
Installed devices:
- default.gaussian (PennyLane-0.21.0.dev0)
- default.mixed (PennyLane-0.21.0.dev0)
- default.qubit (PennyLane-0.21.0.dev0)
- default.qubit.autograd (PennyLane-0.21.0.dev0)
- default.qubit.jax (PennyLane-0.21.0.dev0)
- default.qubit.tf (PennyLane-0.21.0.dev0)
- default.qubit.torch (PennyLane-0.21.0.dev0)
- lightning.qubit (PennyLane-Lightning-0.19.0)
```

☒ I have searched existing GitHub issues to make sure the issue does not already exist.



glassnotes added **bug** on Dec 11, 2021

glassnotes closed this as **completed** on Dec 11, 2021

glassnotes reopened this on Dec 11, 2021

glassnotes added a commit that references this issue on Dec 11, 2021

Fix issue [#2014](#)

dd2a82a

glassnotes mentioned this on Dec 11, 2021

[Fix two-qubit decomp to consider arbitrary number of unitaries #2015](#)

josh146 added a commit that references this issue on Dec 13, 2021

Fix two-qubit decomp to consider arbitrary number of unitaries ([#2015](#) ...)

Verified fd5ee9f

glassnotes closed this as **completed** on Dec 13, 2021

Fix two-qubit decomp to consider arbitrary number of unitaries #2015

<> Code

Merged josh146 merged 3 commits into master from fix_two_qubit_decomp_many_unitaries on Dec 13, 2021

Conversation 8 Commits 3 Checks 0 Files changed 3

+35 -4



glassnotes commented on Dec 11, 2021 • edited

Contributor

Context: Circuits with >3 two-qubit unitaries were yielding a `RecursionError` when put through the `unitary_to_rot` transform.

Description of the Change: Fixes the queuing in the `two_qubit_decomposition` method and adds a relevant unit test.

Benefits: Arbitrary number of two-qubit unitaries can now be decomposed.

Possible Drawbacks: None; although, I'm admittedly not sure why this fixed the problem.

Related GitHub Issues: [#2014](#)



glassnotes added 2 commits 4 years ago

Fix issue #2014

dd2a82a

Add unit test.

9513785



github-actions bot commented on Dec 11, 2021

Contributor

Hello. You may have forgotten to update the changelog!
Please edit [doc/releases/changelog-dev.md](#) with:

- A one-to-two sentence description of the change. You may include a small working example for new features.
- A link back to this PR.
- Your name (or GitHub username) in the contributors section.



Update CHANGELOG.

eff11db

Reviewers

josh146



Assignees

No one assigned

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

2 participants

